UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO (PPGCC)
PROCESSO SELETIVO DE ALUNOS REGULARES 2027 – EDITAL Nº 008/2026

DANILLO GUIMARÃES OLIVEIRA

TESTE E AVALIAÇÃO DE SISTEMAS MULTIAGENTES BASEADOS EM MODELOS DE LINGUAGEM: UMA ABORDAGEM DE ENGENHARIA DE SOFTWARE PARA INTEGRAÇÃO, TRAJETÓRIAS E ROBUSTEZ

Projeto de Pesquisa apresentado ao Programa de Pós-Graduação em Ciência da Computação do Instituto de Informática da Universidade Federal de Goiás (PPGCC/UFG), como requisito parcial para o processo seletivo ao nível de Mestrado Acadêmico.

Linha de Pesquisa: Metodologias e Técnicas de Computação (MTC)
Tema de Pesquisa: Tema MTC3 – Engenharia de Software de Sistemas de Inteligência Artificial
Docente Indicado para Orientação: Prof. Dr. Renato de Freitas Bulcão Neto

Goiânia - GO
Setembro de 2026

1. INTRODUÇÃO

1.1 Contextualização, motivação prática e definição do problema

Modelos de Linguagem de Grande Porte (*Large Language Models* — LLMs) passaram a ser incorporados em diferentes atividades do desenvolvimento de software, abrangendo elicitação de requisitos, síntese de código-fonte, revisão técnica, testes e geração de documentação. Mais recentemente, tais modelos evoluíram para arquiteturas cooperativas compostas por múltiplos agentes especializados, capazes de orquestrar tarefas de maior complexidade cognitiva.

Nessas arquiteturas agênticas, diferentes agentes assumem responsabilidades específicas, como a interpretação de demandas, concepção de soluções arquiteturais, geração de código, consulta a bases de conhecimento (RAG) ou disparo de ferramentas externas. Embora essa organização amplie o potencial de automação, ela introduz severos desafios para os processos de garantia da qualidade, verificação e validação, dado que o comportamento global do sistema passa a depender de inferências probabilísticas e componentes estocásticos.

A literatura de Engenharia de Software para sistemas baseados em Inteligência Artificial documenta expressivas dificuldades relacionadas a requisitos, integração, validação, robustez e operação contínua (Martínez-Fernández et al., 2022; Nahar et al., 2022; Villamizar et al., 2021; Ahmad et al., 2021). Há também uma assimetria relevante apontada pela comunidade acadêmica: a literatura dedica esforços substanciais ao emprego de IA para apoiar atividades de Engenharia de Software ("IA para ES"), enquanto permanece aberta a investigação sobre como princípios, métodos e técnicas formais de Engenharia de Software podem estruturar e avaliar os próprios sistemas de IA ("ES para IA") (Dalpiaz e Niu, 2020).

Essa problemática científica reflete-se diretamente em desafios observados no desenvolvimento de sistemas reais de software.

No âmbito do **Programa de Desenvolvimento de Competências (PDC)**, conduzido pelo **Centro de Excelência em Inteligência Artificial (CEIA)**, foi desenvolvido um sistema multiagente voltado à automação de atividades do ciclo de desenvolvimento de software (SDLC). O sistema foi arquitetado em quatro workflows principais:

1. **Elicitação e Requisitos:** interpretação da demanda de negócio e produção das especificações;
2. **Arquitetura e Design:** definição da decomposição modular e contratos técnicos;
3. **Codificação e Revisão:** síntese, refatoração e inspeção do código-fonte produzido;
4. **Qualidade e Testes:** execução de verificações funcionais e dinâmicas sobre os artefatos gerados.

A coordenação desses workflows foi concebida por meio de um orquestrador determinístico, sem utilização de LLM. Essa camada de controle apresentou comportamento estável e pôde ser verificada utilizando práticas consagradas de Engenharia de Software.

Do mesmo modo, foi possível testar deterministamente diversos componentes de infraestrutura utilizados pelos agentes, tais como schemas Pydantic, parsers, linters, compiladores, analisadores estáticos e ferramentas de apoio. Esses módulos possuíam ampla cobertura de testes unitários e de integração, mantendo comportamento isolado satisfatório.

Entretanto, durante a execução integrada em conjunto com os LLMs sob condições dinâmicas, manifestaram-se anomalias de software que não eram detectadas pelas baterias tradicionais de testes.

Três categorias de falhas destacaram-se com maior frequência:

1. **Variação de comportamento diante de entradas semanticamente equivalentes:** Uma mesma solicitação podia gerar saídas estruturalmente divergentes. Pequenas oscilações léxicas ou inclusões espúrias de formatação (como aspas duplas desnecessárias ou caracteres de escape) eram suficientes para interromper o parsing e paralisar etapas posteriores do fluxo.

2. **Desvios de fluxo em pontos de decisão probabilísticos:** Em nós de controle governados por LLMs, oscilações na classificação semântica conduziam a execução para ramificações incorretas do pipeline, mesmo quando toda a infraestrutura determinística de suporte operava perfeitamente.

3. **Quebra silenciosa de contratos de interface entre agentes:** Respostas contextualmente compreensíveis violavam restrições estruturais implícitas ou pré-condições exigidas pelo workflow subsequente, provocando o efeito cascata de erros acumulados.

Essas evidências práticas demonstraram que testar exclusivamente a infraestrutura determinística é insuficiente para aferir a confiabilidade e a robustez de sistemas multiagentes.

Como resposta experimental preliminar a esse desafio no escopo do PDC/CEIA, estruturou-se uma estratégia de testes organizada em quatro camadas de validação (registrada na Issue #373 e no Pull Request #409 do repositório PDC-ESIA/AI4ES e aplicada como piloto ao pipeline `workflow_coding_review`):

- **Camada 1 — Infraestrutura determinística:** testes unitários e de integração utilizando mocks para isolar chamadas de LLM e verificar schemas, sanitizadores, ferramentas e workspace;

- **Camada 2 — Trajetórias de execução:** captura e asserção de traces estruturados (OpenTelemetry/Langfuse) para observar a ordem de decisão, chamadas de ferramentas e convergência do ciclo iterativo entre agentes sem invocar LLMs reais;

- **Camada 3 — Avaliação semântica:** avaliação de qualidade apoiada em dataset de referência curado, com verificações determinísticas combinadas a avaliadores baseados em LLM (*LLM-as-a-judge*);

- **Camada 4 — Execução segura em sandbox:** execução ponta a ponta em contêineres efêmeros e isolados para verificar o comportamento dinâmico dos artefatos gerados sem

comprometer o ambiente hospedeiro.

Embora tenha trazido ganhos operacionais imediatos, essa experiência prática tratou um caso específico. A literatura científica recente ressalta a necessidade premente de estabelecer fundamentos teóricos e metodológicos gerais para o teste estrutural de agentes, análise de trajetórias e avaliação sistêmica de ecossistemas agênticos (Kohl et al., 2026; Emde et al., 2026; Yehudai et al., 2026). Permanece em aberto como formalizar critérios rigorosos de aceitação, analisar o custo-efetividade e avaliar em que medida essa abordagem pode ser generalizada para outras topologias e domínios de aplicação.

A partir desse cenário, a presente proposta de pesquisa é formulada para responder à seguinte questão central:

> ***Como técnicas de Engenharia de Software podem ser formalizadas, organizadas e combinadas para prover testes de integração confiáveis e avaliar o comportamento sistêmico de sistemas multiagentes baseados em LLMs?***

1.2 Justificativa e Contribuições da Pesquisa

A relevância deste projeto reside na necessidade de construir fundamentos de Engenharia de Software capazes de lidar com a transição de sistemas puramente determinísticos para sistemas estocásticos e agênticos. A experiência vivenciada no PDC/CEIA fornece um cenário real de motivação e experimentação empírica, mas o propósito desta pesquisa transcende a resolução de um problema local, buscando oferecer contribuições científicas e metodológicas mais amplas para a área de Engenharia de Software de Sistemas de Inteligência Artificial (Tema MTC3).

Com base nas lacunas apontadas pela literatura recente, a pesquisa pretende oferecer três contribuições principais:

1. **Contribuição Conceitual e Teórica:** Formalização de um modelo de teste em camadas para sistemas multiagentes, estabelecendo fronteiras claras entre o que pode ser verificado deterministamente, o que exige asserções sobre trajetórias de execução e o que demanda abordagens tolerantes ao não-determinismo, como os Testes Metamórficos (Chen et al., 2018; Kohl et al., 2026). Isso contribui diretamente para o tratamento do problema do oráculo de teste em sistemas baseados em LLMs.
2. **Contribuição Metodológica e Empírica:** Definição de critérios de teste, métricas de avaliação e procedimentos experimentais para mensurar a eficácia de detecção de falhas estocásticas, taxa de falsos alarmes e sobrecarga computacional. O ambiente e os dados derivados do PDC/CEIA serão utilizados como um dos cenários empíricos de validação, permitindo confrontar a teoria com a prática industrial.
3. **Contribuição Tecnológica e Prática:** Desenvolvimento de diretrizes de engenharia e de um protótipo instrumentalizado de código aberto que permita a equipes de software incorporar testes estruturais e semânticos em esteiras de integração contínua (CI/CD), prevenindo regressões comportamentais silenciosas antes da implantação em produção.

O projeto dialoga diretamente com as prioridades do PPGCC/UFG e com a linha de pesquisa do **Prof. Dr. Renato de Freitas Bulcão Neto**, cuja liderança em Engenharia de Software Empírica, qualidade e processos de software fornece a base ideal para orientar a condução deste trabalho.

1.3 Objetivos

1.3.1 Objetivo Geral

Conceber, formalizar e avaliar experimentalmente uma abordagem de Engenharia de Software organizada em camadas para teste e avaliação de sistemas multiagentes baseados em LLMs, integrando verificações determinísticas, análise de trajetórias de execução, avaliação semântica e execução segura.

1.3.2 Objetivos Específicos

1. **Revisar sistematicamente a literatura** sobre teste e avaliação de sistemas baseados em LLMs e sistemas multiagentes, mapeando modos de falha, práticas emergentes de observabilidade e limitações das abordagens correntes.
2. **Formalizar o modelo conceitual de teste em quatro camadas**, especificando objetivos, classes de defeitos visadas, propriedades analisadas e limitações de cada nível de abstração.
3. **Modelar mecanismos de asserção sobre trajetórias de execução**, utilizando telemetria e traces para verificar ordem de transições, chamadas de ferramentas e estabilidade de ciclos iterativos entre agentes.
4. **Investigar e definir estratégias para avaliação de comportamento estocástico**, explorando a aplicação de relações metamórficas e contratos semânticos para cenários sem oráculo pontual estrito.
5. **Desenvolver um protótipo de software de suporte aos testes**, instrumentalizando pipelines de execução e viabilizando a automação das camadas de verificação.
6. **Avaliar empiricamente a abordagem proposta** por meio de estudos de caso e experimentos controlados, utilizando o ecossistema do PDC/CEIA como cenário piloto de experimentação, mensurando capacidade de detecção de falhas, taxas de falsos positivos e custos computacionais.

2. REVISÃO DA LITERATURA

A revisão da literatura está estruturada em quatro pilares conceituais diretamente vinculados ao problema e às contribuições pretendidas.

2.1 Engenharia de Software para Sistemas de IA

A incorporação de modelos de aprendizado e LLMs redefine o ciclo de desenvolvimento de software. Martínez-Fernández et al. (2022) apresentam uma visão abrangente dos desafios de ES para sistemas baseados em IA, destacando gargalos de rastreabilidade, robustez e ciclo de vida não-linear. Complementarmente, Nahar et al. (2022) analisam no ICSE os desafios de colaboração interdisciplinar, documentação e processos de engenharia, ressaltando que módulos estatísticos e ferramentas determinísticas costumam ser tratados em silos isolados, provocando falhas graves durante a integração sistêmica.

Esses diagnósticos fundamentam a necessidade de abordagens estruturadas de Engenharia de Software voltadas a governar as interações e decisões desses sistemas.

2.2 Requisitos, Contratos e Qualidade em Sistemas Não Determinísticos

Definir o comportamento esperado em sistemas generativos impõe desafios à engenharia tradicional. Villamizar et al. (2021) evidenciam a escassez de métodos formais para especificação e teste de requisitos em sistemas orientados a aprendizado de máquina. Ahmad et al. (2021) detalham as dificuldades de validação e tolerância a falhas quando o comportamento operacional diverge do domínio de treinamento. Dalpiaz e Niu (2020) destacam a urgência de conceber especificações probabilísticas e adaptativas, capazes de estabelecer contratos de interface operacionais mesmo sob incerteza comportamental.

2.3 Trajetórias de Execução, Traces e Testes Metamórficos

Em sistemas de software convencionais, a verificação apoia-se na comparação estrita entre a saída gerada e o resultado esperado por um oráculo. Em agentes baseados em LLMs, a multiplicidade de respostas semanticamente válidas inviabiliza asserções sintáticas fechadas (*test oracle problem*).

Nesse contexto, Kohl et al. (2026) introduzem a noção de teste estrutural para agentes de LLM, utilizando traces de execução (OpenTelemetry) para auditar decisões intermediárias e chamadas de ferramentas. Essa perspectiva permite identificar falhas latentes que não transparecem imediatamente na saída textual final.

De forma complementar, Chen et al. (1998, 2018) estabeleceram a teoria dos **Testes Metamórficos (Metamorphic Testing – MT)** como solução rigorosa para programas sem oráculos exatos. O teste metamórfico fundamenta-se na verificação de **Relações Metamórficas (MRs)** entre múltiplas execuções correlacionadas. Em agentes de LLM, se uma perturbação preservadora de sentido é introduzida na entrada e o comportamento resultante quebra uma invariante lógica ou semântica, identifica-se a ocorrência de um defeito no sistema.

2.4 Avaliação Sistêmica de Sistemas Multiagentes

A avaliação de agentes de IA transcende a aferição isolada de modelos fundacionais. No survey da ACL, Yehudai et al. (2026) sintetizam métodos e benchmarks existentes, identificando lacunas críticas na mensuração de robustez, custo-eficiência e estratégias escaláveis de avaliação contínua. Em sintonia, Emde et al. (2026) apresentam o MASEval (ACL 2026), demonstrando experimentalmente que decisões no nível de sistema de software — como topologias de comunicação, mecanismos de harness e lógica de tratamento de erros — impactam o desempenho e a confiabilidade de maneira tão determinante quanto a escolha do modelo neural.

Essas evidências reforçam que a unidade de análise e teste deve ser o sistema multiagente integrado em sua completude de software.

3. PROPOSTA DE PESQUISA E MÉTODO

3.1 Abordagem Metodológica

A pesquisa possui natureza aplicada e adotará como referencial inicial princípios da **Design Science Research (DSR)** articulados a métodos da **Engenharia de Software Experimental** (Wazlawick, 2010). Esse enquadramento epistemológico é adequado para a formulação conceitual, implementação de protótipos e validação empírica rigorosa de artefatos que resolvem problemas reais de engenharia.

O delineamento experimental específico será calibrado a partir dos resultados da revisão bibliográfica e da caracterização dos modos de falha.

3.2 Fases do Método de Pesquisa

Fase 1: Revisão da Literatura e Caracterização do Problema (Meses 1 a 4) Execução de um levantamento estruturado em bases internacionais (IEEE Xplore, ACM DL, Scopus) para categorizar modos de falha em sistemas multiagentes, estratégias de asserção em trajetórias e aplicações de testes metamórficos, consolidando uma taxonomia teórica de falhas agênticas.

Fase 2: Formalização Conceitual da Abordagem em Quatro Camadas (Meses 5 a 10) Estruturação formal do modelo de quatro camadas: especificação das propriedades das Camadas 1 e 2 (mocks herméticos e asserções programáticas em traces de telemetria) e modelagem conceitual das Camadas 3 e 4 (relações metamórficas, contratos semânticos, calibração de avaliadores automatizados e sandboxing).

Fase 3: Desenvolvimento do Protótipo de Suporte aos Testes (Meses 11 a 14) Construção de uma ferramenta protótipo em código aberto voltada a apoiar a instrumentação e execução automatizada das estratégias de teste concebidas, integrando analisadores de traces, orquestrador de testes metamórficos e runners seletivos para esteiras de integração contínua (CI/CD).

Fase 4: Avaliação Empírica e Validação Experimental (Meses 15 a 19) Planejamento e condução de experimentos empíricos com sistemas multiagentes. O ecossistema do PDC/CEIA (especialmente o pipeline piloto de codificação e revisão) será empregado como cenário real de experimentação, comparando a abordagem proposta com as práticas tradicionais de teste. Variáveis de resposta incluirão: eficácia na revelação de falhas estocásticas, taxa de falsos alarmes e sobrecarga de tempo e custo computacional.

Fase 5: Consolidação dos Resultados e Redação da Dissertação (Meses 20 a 24) Sistematização teórica e análise estatística dos dados empíricos na dissertação de mestrado, com redação e submissão de artigos científicos para conferências qualificadas (ex.: SBES, ICSE) e periódicos da área.

3.3 Plano de Trabalho e Cronograma Físico de Execução (24 Meses)

A distribuição das etapas ao longo dos doze bimestres regulares (B1 a B12) do curso é apresentada a seguir:

Atividade / Bimestre	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12
1. Disciplinas Obrigatórias e Eletivas do PPGCC	X	X	X	X								
2. Revisão da literatura e taxonomia de falhas	X	X										
3. Formalização da abordagem em camadas e MRs			X	X	X							
4. Engenharia e desenvolvimento do protótipo					X	X	X					
5. Exame de Qualificação do Mestrado						X						

6. Condução dos experimentos e avaliação empírica							X	X	X			
7. Análise de resultados e redação de artigos								X	X	X		
8. Redação final da dissertação e defesa pública										X	X	X

3.4 Viabilidade de Realização e Infraestrutura

A viabilidade da pesquisa ampara-se na convergência de infraestrutura computacional e suporte metodológico: - **Infraestrutura Computacional do INF/UFG:** Acesso aos laboratórios e clusters do Instituto de Informática da UFG para execução local e controlada de modelos de linguagem de código aberto (LLaMA-3, Mistral), assegurando repetibilidade experimental e independência de APIs comerciais pagas. - **Disponibilidade de Cenários e Artefatos Reais:** O acesso aos workflows, dados e histórico de desenvolvimento do PDC/CEIA viabiliza um ambiente de experimentação empírica rico e já instrumentalizado. - **Supervisão Acadêmica:** A orientação pelo **Prof. Dr. Renato de Freitas Bulcão Neto** oferece rigor metodológico e respaldo científico nas áreas de Engenharia de Software Empírica, qualidade de código e processos de software.

4. REFERÊNCIAS BIBLIOGRÁFICAS

1. AHMAD, K.; BANO, M.; ABDELRAZEK, M.; ARORA, C.; GRUNDY, J. What's up with requirements engineering for artificial intelligence systems? In: **Proceedings of the 29th IEEE International Requirements Engineering Conference (RE 2021)**. IEEE, 2021. p. 1–12.

2. CHEN, T. Y.; CHEUNG, S. C.; YIU, S. M. Metamorphic testing: a new approach for generating next test cases. **Technical Report HKUST-CS98-01**. Hong Kong University of Science and Technology, 1998.

3. CHEN, T. Y.; KUO, F. C.; KOWSARI, R. et al. Metamorphic testing: a review of challenges and opportunities. **ACM Computing Surveys**, v. 51, n. 1, p. 1–27, 2018.

4. DALPIAZ, F.; NIU, N. Requirements engineering in the days of artificial intelligence. **IEEE Software**, v. 37, n. 4, p. 7–10, 2020.

5. EMDE, C.; RUBINSTEIN, A.; GOEL, A.; HEAKL, A.; YUN, S.; OH, S. J.; GUBRI, M. MASEval: Extending Multi-Agent Evaluation from Models to Systems. In: **Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)**. 2026. p. 345–356.

6. KOHL, J.; KRUSE, O.; MOSTAFA, Y.; LUCKOW, A.; SCHROER, K. et al. Automated structural testing of LLM-based agents: methods, framework, and case studies. **arXiv preprint arXiv:2601.18827**, 2026.

7. MARTÍNEZ-FERNÁNDEZ, S.; BOGNER, J.; FRANCH, X.; ORIOL, M.; SIEBERT, J.; TRENDOWICZ, A.; VOLLMER, A. M.; WAGNER, S. Software engineering for AI-based systems: a survey. **ACM Transactions on Software Engineering and Methodology**, v. 31, n. 2, p. 1–59, 2022.

8. NAHAR, N.; ZHOU, S.; LEWIS, G.; KASTNER, C. Collaboration challenges in building ML-enabled systems: communication, documentation, engineering, and process. In: **Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)**. New York: ACM, 2022. p. 413–425.

9. VILLAMIZAR, H.; ESCOVEDO, T.; KALINOWSKI, M. Requirements engineering for machine learning: a systematic mapping study. In: **2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)**. IEEE, 2021. p. 29–36.

10. WAZLAWICK, Raul Sidnei. Uma reflexão sobre a pesquisa em Ciência da Computação à luz da classificação das ciências e do método científico. **Revista de Sistemas de Informação da FSMA**, Macaé, n. 6, p. 3–10, 2010.

11. YEHUDAI, A.; EDEN, L.; LI, A.; UZIEL, G.; ZHAO, Y.; BAR-HAIM, R.; COHAN, A.; SHMUELI-SCHEUER, M. A survey on evaluation of LLM-based agents. In: **Findings of the Association for Computational Linguistics: ACL 2026**. 2026. p. 26690–26714.